

Braillix simulator - second brutal review

Reviewed 2026-09-16, against the state after the hotspot, electronics, pod and hardware-link work. Supersedes `SIMULATOR_REVIEW.md`, which was written before any of that existed.

Measured, not guessed: payload from `du`, font sizes and panel counts from `index.html`, everything else read out of the running page.

The one-sentence verdict

The engineering has outrun the presentation, and the gap is now wider than it was in August. There is more to look at, more to control, and still nothing that tells a newcomer what they are looking at or what to do first.

What is genuinely strong

State these out loud; they are unusual and they are earned.

- **Every constant is parsed out of the CAD.** The simulator cannot silently disagree with the design, and the extractor now resolves build options rather than guessing.
 - **It caught real hardware bugs** - the 5.2mm shaft clash, and the stale-GLB/Gray-order mismatch. A simulator that finds design faults has justified itself.
 - **The same page drives the real cell** over USB or Bluetooth, at the motor's actual trapezoidal speed profile, waiting for the hardware to arrive before advancing. That is a genuinely impressive thing to demonstrate.
 - **No build step.** One HTML file, three JS files, Three.js vendored. It will still open in 2031.
 - Braille encoding is correct including indicators, which most implementations get wrong.
-

Visual

1. First load is a featureless grey brick

Seven of the interesting objects - `cam`, `comb`, `linkage_1`, `uln2003`, `stepper`, `hall`, `esp32` - are **sealed inside an opaque box**. With zero clicks you see a closed enclosure and a spinning nothing.

The single best change available: **start with X-Ray on**. The mechanism is the point; the walls are not. Someone who wants the solid exterior can switch it off.

2. The text is too small to read anywhere but your own laptop

Font size	Declarations
<code><= 12px</code>	14 of 21
10px or below	8

Body text runs 10-11px in `--dim` (`#8d93b0`) on `--panel` (`#181a2b`) - about **5.3:1**. Fine at 40cm. Unreadable on a projector, on a phone at arm's length, or by anyone over about forty.

3. Nine fixed panels fighting for four corners

`#brand`, `#cell`, `#ui`, `#decode`, `#info`, `#spots`, `#hint`, `#loading`, `#stage`. At 1280x720 this is busy. On a laptop at 1366x768 it is crowded. There is no hierarchy - everything is the same card, the same border, the same weight.

4. 3.8 MB and a spinner with no number

<code>braillix.glb</code>	2570 KB	<code>vendor/</code>	943 KB
<code>pod.glb</code>	153 KB	<code>motor.glb</code>	91 KB
TOTAL	3842 KB		

On mobile data that is a long blank screen. Draco typically cuts the GLB 70-90% and the loader is already vendored.

Functional

5. Hotspot dots have no occlusion test

`updateHotspots()` projects each anchor and places a dot. There is **no raycast**, so a dot for the ULN2003 renders happily on top of the solid box wall hiding it. You click a glowing dot and get a description of something you cannot see. Fix: raycast anchor->camera, hide the dot when something opaque is in the way. ~10 lines.

6. Two of seven buttons serve almost no one

Connect USB and Connect Bluetooth occupy 29% of the control panel. **Almost every visitor has no Braillix hardware.** They are the most impressive feature and the least usable one, and right now they are equal citizens with Reset View.

Move them behind a "Have the hardware?" disclosure and give the space back.

7. No keyboard control

One keydown handler in the whole app, and it only handles Escape. No space to pause, no arrows to step, no digits to jump. Every interaction requires precise mouse work on a 15px target.

8. The animation autoplays

`running = true` at load. Things are moving before the viewer has oriented, and the first thing many people do is hunt for the stop button. **Start paused**, showing one settled letter.

9. Smaller functional gaps

- Speed slider reads "1.0x" - of *what*? Label it in characters/minute.
- No way to return to the first-run state without reloading.
- Reset View restores the camera but not X-Ray, Electronics, or the open info card.
- Failure paths degrade silently: a missing `pod.glb` logs to console and quietly draws

worse geometry. Nobody sees a console on a deployed site.

Experience and comprehension

10. Nothing says what Braillix is

The entire explanation is six words: **"Refreshable Braille Cell - Mechanism Simulator"**.

There is no statement of what a refreshable braille display is, that commercial ones cost lakhs, what this one costs, or that one cell is a teaching device while the product is a row of them. **The model answers "how". Nobody has answered "why".**

11. The only onboarding is a mouse tutorial

`drag to orbit - scroll to zoom - right-drag to pan`

That teaches the camera, not the machine. A first-time visitor does not know what a cam is, why a disc has rings on it, or what the six circles in the corner mean.

12. The decode panel is jargon

`state 12/63   step 800/4096   bits 001100   4.22deg`

This is the cleverest part of the project and it is written for someone who already understands it. To a newcomer it is noise. It needs one line of plain language above it - **"the cam turns to one of 64 angles; each angle raises a different set of dots"**.

13. No sense of scale

Nothing indicates the cell is 68mm. It could be a matchbox or a washing machine. A dimension line or a fingertip model would make the 2.6mm braille pitch mean something.

14. Attribution is leaking into the UI

One hotspot is titled:

The credit is required, the placement is not. Put it in an About panel, keep the label clean.

Accessibility - still the sharpest criticism available

Check	Result
ARIA attributes	0
Keyboard handlers	1 (Escape)
Audio output	none
Text alternative for the braille readout	none

A project about braille that a blind person cannot use. The six-dot readout is coloured <div>s with no accessible name. Every hotspot is a <button> with a title and no label.

This is also the cheapest big win here: button labels, aria-live on the letter readout, arrow-key stepping, and the Web Speech API for the current character is **one day** and it turns the project's weakest point into a talking point.

Ranked - biggest gain per hour

#	Change	Effort	Why
1	Start with X-Ray on and paused	15 min	Turns a grey brick into the actual mechanism, instantly
2	Plain-language line above the decode panel	30 min	Makes the cleverest part legible
3	Occlusion raycast on hotspots	1 h	Stops dots pointing at things you cannot see
4	Bump type size and contrast	1 h	Readable off your own screen
5	Accessibility pass	1 day	Removes the one criticism with no good answer
6	Hide the hardware buttons behind a disclosure	1 h	Gives 29% of the panel back
7	A "what is this" intro card, dismissible	3 h	Answers *why* before *how*
8	Keyboard shortcuts	30 min	Space, arrows, digits, R, X
9	Scale reference	2 h	Makes 68mm mean something
10	Draco-compress the GLB	1 h	2.5 MB down to roughly 400 KB

Items 1-4 total about three hours and change the first impression completely.

Deliberately not worth doing

- **More electronics detail.** Diminishing returns; nobody has ever asked.
- **Photorealism.** The render already reads as a real object.
- **A React rewrite.** The no-build-step property is a real asset. Do not trade it.
- **Fixing the 5.2mm shaft clash in the sim.** It is a hardware problem. Label it and

present it as evidence the simulator works.

The honest summary

The simulator is now a genuinely capable instrument that can drive real hardware and has caught real design faults. It is still presented as if the viewer already knows the project. **Three hours on first impression - X-ray by default, paused, plain language, readable type - would do more for it than another month of geometry.**